Project Interim Report

On

Online Food Delivery System

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Vishal Kumar

ERP: RU-25-11609

Bachelor of Technology in Computer Science & Engineering in association with GOOGLE

Under the Guidance of

Mr. Harsh Multani

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Sr.no Topic

1. Abstract
2. Introduction
3. Scope
4. System Requirements
5. Methodology
6. Class diagram
7. Implementation
8. Sample input/output
9. Future Enhancement
10. Gantt chart
11. Conclusion
12. References

Abstract

The Online Food Delivery System is a console-based Java application developed using Object-Oriented Programming (OOP) principles. It simulates the core functionality of modern food delivery platforms such as Swiggy and Zomato, allowing users to browse a restaurant's menu, select multiple food items with quantities, and receive a complete order summary with an itemized bill.

The system is structured around three interacting classes — FoodItem, Restaurant, and Order. The FoodItem class encapsulates the name and price of individual menu items. The Restaurant class stores the restaurant's name and its menu as an array of FoodItem objects. The Order class manages the items selected by the user, calculates the subtotal based on item price and quantity, applies delivery charges based on a threshold rule, computes 5% GST tax, and generates a complete bill summary.

This project demonstrates the practical application of key OOP concepts including encapsulation, object composition, array-based data management, and method-driven billing logic. It provides students with hands-on experience in designing multi-class Java systems that model real-world business processes in a structured and modular way.

Introduction

The rapid growth of online food ordering has transformed the way people access restaurant services. Platforms like Zomato, Swiggy, and Uber Eats handle millions of orders daily by managing restaurants, menus, customer selections, and billing in real time. Building a simplified version of such a system using Java and OOP principles provides valuable insight into how real-world software models interconnected entities.

The Online Food Delivery System is a console-based Java application developed as part of the Object Oriented Programming with Java course (RU-100-01-00012) at Rungta International Skills University. The system allows a restaurant to list food items on a menu, enables a user to browse that menu, select items with desired quantities, and receive a detailed order bill including subtotal, delivery charges, and GST tax.

The application uses three core classes — FoodItem, Restaurant, and Order — which interact through object composition and array management. This design reflects how real systems separate concerns: one class handles item data, another manages restaurant details, and a third handles order logic and billing.

Key features of the application include:

- Restaurant menu listing with item names and prices
- User selection of multiple food items with quantities
- Subtotal calculation as sum of price multiplied by quantity for each item
- Smart delivery charge: free for orders above Rs. 500, else Rs. 50 flat
- 5% GST tax applied on the subtotal
- Complete order summary with itemized bill and final total

Objectives

The primary goal of this project is to build a functional Online Food Delivery System that demonstrates Object-Oriented Programming in a real-world billing context. The specific objectives are:

- Model real-world food delivery entities — restaurants, food items, and orders — as Java classes
- Design a FoodItem class to encapsulate item name and price using private attributes and getters
- Design a Restaurant class to store a restaurant's name and menu array of FoodItem objects
- Design an Order class to manage selected items and their quantities using parallel arrays

- Implement an addItem() method to add food items along with their quantities to an order
- Implement billing logic: subtotal as sum of (price x quantity) for all selected items
- Apply a conditional delivery charge: Rs. 0 if subtotal exceeds Rs. 500, else Rs. 50
- Calculate 5% GST tax on the subtotal and add it to arrive at the final total
- Display a complete and well-formatted order summary to the user
- Apply OOP principles: encapsulation, object composition, array handling, and method separation

Scope

Current Scope

The current version is a console-based Java application that models a single-restaurant food ordering and billing system. Its scope includes:

- Representing a single restaurant with a fixed menu of FoodItem objects
- Allowing selection of multiple food items with specified quantities
- Computing subtotal, delivery charge, tax (5% GST), and final total
- Displaying a clean, itemized order summary on the console

- Enforcing delivery charge threshold: free delivery for orders above Rs. 500
- Using arrays (not ArrayList) to store menu items and order selections

Future Scope

The system can be significantly extended in future iterations:

- Multiple Restaurants: Allow users to browse and order from multiple restaurant menus in a single session
- User Input via Scanner: Make item selection dynamic by reading user choices from keyboard input
- Database Integration: Store restaurant and menu data in a MySQL database for real-time management
- Distance-Based Delivery Charge: Calculate delivery fees based on user location and restaurant distance
- Discount Coupons: Apply promo codes or percentage-based discounts to the order total
- Order History: Maintain a log of all past orders per user session
- GUI Interface: Build a visual ordering interface using Java Swing or JavaFX
- Web Application: Deploy as a REST API-based web service using Java Spring Boot

System Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	500 MB free disk space
Display	1024 x 768 resolution or higher

Software Requirements

Software	Details
Operating System	Windows 10 / macOS / Linux (Ubuntu)
Java Development Kit	JDK 8 or above
IDE / Editor	VS Code / Eclipse / IntelliJ IDEA / Edit Plus
Compiler	javac (bundled with JDK)
Terminal / Console	Command Prompt / PowerShell / Terminal

Methodology

The Online Food Delivery System follows a clear sequential workflow from menu setup to bill generation. The complete methodology is described below:

1. FoodItem Initialization: FoodItem objects are created with a name and price. These represent individual dishes available in the restaurant's menu such as Burger (Rs. 100), Pizza (Rs. 300), and Pasta (Rs. 200).
2. Restaurant Setup: A Restaurant object is created with a name and an array of FoodItem objects as its menu. The displayMenu() method prints all available items with their prices in a formatted list.
3. Order Creation: An Order object is created to hold the customer's selections. It maintains two parallel arrays — one for FoodItem objects and one for integer quantities — along with a counter tracking how many distinct items have been added.
4. Adding Items: The addItem(FoodItem item, int quantity) method stores the selected FoodItem and its quantity at the next available index in the order arrays. This simulates a user adding items to a cart.
5. Subtotal Calculation: The calculateTotal() method iterates through all added items, multiplying each item's price by its quantity, and sums these values to produce the subtotal.

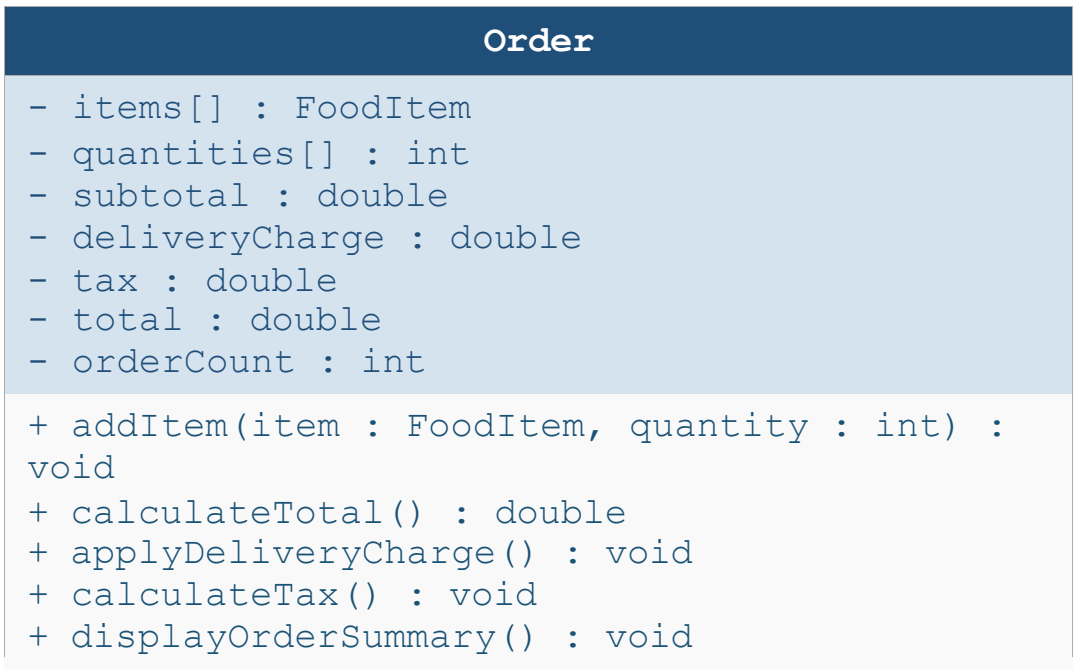
6. Delivery Charge Logic: If the subtotal is greater than Rs. 500, delivery is free (Rs. 0). Otherwise, a flat delivery charge of Rs. 50 is applied.
7. Tax Calculation: 5% GST is calculated on the subtotal (tax = subtotal x 0.05).
8. Final Total and Summary: The total is computed as subtotal + deliveryCharge + tax. The displayOrderSummary() method prints each item with its quantity and line total, followed by subtotal, delivery charge, tax, and the final payable amount.

Class Diagram

The system is built using three interacting Java classes. Their UML-style class diagram is shown below, including all attributes and methods:

FoodItem
- itemName : String - price : double
+ FoodItem(itemName : String, price : double) + getItemName() : String + getPrice() : double + displayItem() : void

Restaurant
- name : String - menu[] : FoodItem
+ Restaurant(name : String, menu[] : FoodItem) + getName() : String + getMenu() : FoodItem[] + displayMenu() : void



Relationships: Restaurant contains an array of FoodItem objects (composition). Order references FoodItem objects selected by the user alongside their quantities (association).

Implementation

The system is implemented using three Java classes. The logic of each class is explained below (source code is not pasted here — refer to the submitted Java file):

1. FoodItem Class

The FoodItem class represents a single dish available in a restaurant's menu. It stores two private attributes — itemName (a String) and price (a double). A parameterized constructor initializes both fields when a FoodItem object is created. Public getter methods

getItemName() and getPrice() allow other classes to access these values. A displayItem() method prints the item name and formatted price to the console. This class follows encapsulation strictly — no attribute is publicly accessible directly.

2. Restaurant Class

The Restaurant class represents a food outlet offering a menu of dishes. It stores the restaurant's name as a String and its menu as an array of FoodItem objects. A constructor initializes both fields. The getMenu() method returns the full menu array, and the displayMenu() method iterates through all FoodItem objects and prints each item's name and price in a numbered format, giving users a clear view of available dishes.

3. Order Class

The Order class is the core of the billing system. It maintains two parallel arrays — one for FoodItem references and one for integer quantities — along with a counter orderCount tracking how many distinct items have been added. The addItem() method stores each selected item and its quantity at the current index. The calculateTotal() method loops through all items, multiplying each item's price by its quantity to build the subtotal. The applyDeliveryCharge() method checks whether the subtotal exceeds Rs. 500 — if yes, delivery is free; otherwise Rs. 50 is charged. The calculateTax() method computes 5% GST on the subtotal. Finally,

displayOrderSummary() prints each item with quantity, unit price, and line total, followed by the subtotal, delivery charge, tax, and grand total in a clean, formatted layout.

Sample Input / Output

The following terminal output demonstrates a complete ordering session — menu display, item selection, and final bill generation:

```
=== Welcome to QuickBite Food Delivery ===

Restaurant: QuickBite Kitchen
--- Menu ---
  1. Burger        Rs. 100.00
  2. Pizza         Rs. 300.00
  3. Pasta         Rs. 200.00
  4. Cold Drink    Rs.  50.00

>> Adding: Burger x2, Pizza x1 to order...

=========== ORDER SUMMARY ===========
  Burger         x2   =  Rs. 200.00
  Pizza          x1   =  Rs. 300.00
-------------------------------------------------------
  Subtotal             :  Rs. 500.00
  Delivery Charge      :  Rs.   0.00  (Free!)
  Tax (5% GST)         :  Rs.  25.00
-------------------------------------------------------
  TOTAL AMOUNT         :  Rs. 525.00
=======================================
  Thank you for ordering from QuickBite!
```

Future Enhancements

The current implementation provides a solid foundation for a food delivery billing system. The following enhancements are proposed for future development:

- Multiple Restaurant Support: Allow users to browse and order from multiple restaurants in a single session, with each restaurant maintaining its own menu and the order combining items across restaurants.
- Dynamic User Input via Scanner: Replace hardcoded item selection with real-time keyboard input, allowing users to interactively choose items and quantities from the displayed menu.
- Distance-Based Delivery Charge: Calculate delivery fees dynamically based on the distance between the restaurant and the user's address, replacing the fixed threshold rule.
- Discount Coupons: Implement a coupon system where users can enter promo codes (e.g., SAVE10 for 10% off) that are validated and applied to the subtotal before tax.
- Order History Tracking: Maintain a log of all completed orders in the session, allowing users to review previous orders and reorder from history.
- Database Integration: Store restaurant menus, orders, and user data in a MySQL database using JDBC, enabling persistent data management beyond a single session.

- GUI Interface: Develop a graphical food ordering interface using Java Swing or JavaFX with visual menus, cart management, and a receipt display.
- Rating System: Allow users to rate food items after delivery, storing feedback for each item to improve menu recommendations.

Gantt Chart

The following Gantt chart outlines the project development timeline across a 4-week period:

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis & Design	■			
Class Design & UML Diagram	■	■		
Coding — FoodItem Class		■		
Coding — Restaurant Class		■		
Coding — Order & Billing Logic		■	■	
Delivery Charge & Tax Logic			■	
Order Summary Display			■	
Testing & Debugging			■	■
Report Writing			■	■
Final Review & Submission				■

■ = Active / In Progress

Conclusion

The Online Food Delivery System successfully demonstrates the practical application of Object-Oriented Programming concepts in Java through a real-world billing scenario. By modeling three distinct entities — food items, restaurants, and orders — as separate interacting classes, the project follows a clean, modular architecture that accurately reflects the structure of actual food delivery platforms.

The project illustrates key OOP principles in action. Encapsulation is achieved in the FoodItem class by keeping itemName and price private with public getters. Object composition is demonstrated in the Restaurant class which contains an array of FoodItem objects, and in the Order class which references selected FoodItem objects alongside quantities. Method separation ensures that billing responsibilities — subtotal calculation, delivery charge logic, and tax computation — are cleanly divided into individual methods rather than bundled into a single block.

The billing logic itself reflects real-world business rules: a conditional delivery charge (free above Rs. 500, else Rs. 50 flat) and a 5% GST applied on the subtotal. The final order summary presents a professional, itemized receipt that mirrors what users see on platforms like Zomato and Swiggy.

Through this project, students strengthen their ability to design interacting classes, manage object arrays, implement conditional business logic, and produce formatted output in Java. The system establishes a strong foundation for future development including dynamic user input, multi-restaurant support, database integration, and GUI-based food ordering applications.

References

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

[2] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River: Prentice Hall, 2006.

[3] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

[4] GeeksforGeeks, "Object Oriented Programming in Java," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/

[5] TutorialsPoint, "Java OOPs Concepts," TutorialsPoint. [Online]. Available: https://www.tutorialspoint.com/java/java_oops_concepts.htm

[6] Oracle, "Java Arrays," Oracle Java Tutorials. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

[7] R. Sharma and P. Verma, "Object-Oriented Design for E-Commerce Systems," International Journal of Software Engineering, vol. 11, no. 2, pp. 67-74, 2023.